RHEMLO

From demo to live server

A step-by-step deployment guide

Vercel · Google · AWS · Supabase

Version 2 · November 2026

Swipe to turn the page

START HERE

Contents

How to use this guide · 3
What Rhemlo is today · 4
The pieces of a live Rhemlo · 5
Choose your path · 6
Accounts and tools checklist · 7
Put the demo online (Vercel) · 8
Check the public demo · 9
The live backend is built and tested · 10
What runs where · 11
Create the Supabase projects · 12
Run the migrations · 13
Load the manual content · 14
Staff, coaches and classes · 15
Sign-in settings · 16
Switch the app to live · 17
Staging test checklist · 18
Host on Firebase Hosting · 19
Build and deploy on Firebase · 20
Host on AWS Amplify · 21
AWS rewrites and domain · 22
Your domain and HTTPS · 23
Email that arrives · 24
Security and privacy checklist · 25
Monitoring · 26
Rough monthly costs · 27
Go-live checklist · 28
Rolling back · 29
The journey from here · 30
Glossary · 31

HOW TO READ THIS

How to use this guide

This guide takes Rhemlo from the demo it is today to a live service that real learners sign in to. Read it once end to end, then work through it page by page.

- **Swipe or tap** to move between pages. Each page is one idea or one set of steps.
- **Contents** (page 2) and your PDF app's bookmarks jump straight to any section. The footer link on every page returns to Contents.
- **Grey boxes** are commands or settings to copy exactly.
- **Coloured boxes**: purple is a tip, orange is a caution, red is a must-do before real users, green is the finish line of a phase.

> Prices, plan limits and console screens change often. Where this guide gives a price or a menu name, check it on the provider's website on the day you do it.

The three phases

Phase	What you get	Effort
0 · Public demo	Rhemlo on a real web address, still on the demo server	An afternoon
1 · Live backend	Real accounts, sync across devices, real leagues and community	The real work: weeks
2 · Go live	Domain, email, security, monitoring, launch day	A few days

WHERE YOU ARE

What Rhemlo is today

The app is a **static web app** (React + Vite). Building it produces a folder, `dist`, of plain files: pages, scripts, images and the manual data. Any static host can serve it.

Already built

- All study features, the CBT mock, planner, insights, community, admin and steward screens.
- The manuals as files: `public/data` (about 32 MB) and artwork in `public/assets` (about 76 MB).
- A service worker for offline use (on the published site only).
- **A demo server** that runs inside each browser, with demo accounts and simulated people.
- **The live backend**: seven database migrations in `supabase/migrations` and the app's live adapters, switched on by three settings.

> Until you switch to live, the demo server keeps everything in each visitor's own browser. Phase 1 puts the real backend in place.

THE BIG PICTURE

The pieces of a live Rhemlo

Piece	Job	Recommended
Web host + CDN	Serves the app files fast, worldwide, over HTTPS	Vercel (Google and AWS options later)
Database + API	Accounts, progress, leagues, chat, moderation	Supabase (managed Postgres)
Sign-in	Passwords, email links, sessions	Supabase Auth
Scheduled jobs	Weekly league results, clean-ups	Postgres pg_cron inside Supabase
Email	Sign-in links, consent requests, reports	Resend or Postmark via SMTP
Domain	Your own address, e.g. rhemlo.app	Any registrar
Monitoring	Errors and uptime	Sentry + an uptime checker

The browser talks to Supabase directly with a public “anon” key. Security comes from **row-level security** rules inside the database, which the bundle’s migrations already define.

DECISION

Choose your path

The app's live adapter speaks Supabase's API (`/auth/v1` for sign-in and `/rest/v1/rpc` for database functions). So keep **Supabase as the backend on every path**. Only the web host changes.

Path	Web host	Backend	Choose it if…
A · best fit	Vercel	Supabase Cloud	You want the fastest, simplest route. The repo already has vercel.json.
B · Google	Firebase Hosting	Supabase Cloud	Your organisation standardises on Google Cloud.
C · AWS	Amplify Hosting	Supabase Cloud	Your organisation standardises on AWS.

Moving the database itself to Google Cloud SQL or AWS RDS means replacing Supabase Auth and its API layer: a large rewrite. Supabase can be self-hosted on a Google or AWS server later if data must live in your own cloud account.

Pick a Supabase region close to your learners (for Nigeria, a European region such as London is usually closest). Region cannot be changed later without moving the project.

BEFORE YOU START

Accounts and tools checklist

Accounts (free to open)

- **GitHub**: the code is at github.com/tiwaomotosho/rhemlo_app.
- **Vercel**: sign up with GitHub. Or Firebase / AWS for paths B and C.
- **Supabase**: sign up with GitHub.
- **Domain registrar**: Namecheap, Cloudflare, or the one your church already uses.
- **Email sender**: Resend or Postmark.
- **Sentry** (optional): error reports.

Tools on your computer

- Git and Bun (already installed for this project).
- Node.js LTS, needed for the Supabase CLI via npx.
- For path B: the Firebase CLI. For path C: nothing extra, as Amplify builds in the cloud.

> Use one shared “owner” email (for example tech@yourchurch.org) for every service, with two-factor sign-in turned on. Never tie the service to one person’s private email.

PHASE 0 · AN AFTERNOON

Put the demo online (Vercel)

This makes Rhemlo reachable at a real address today, still on the demo server. It is perfect for testers and stakeholders.

1 Push your latest code to GitHub

```
git push origin master
```

2 Import the project in Vercel

- vercel.com → **Add New…** → **Project** → pick `rhemlo_app`.
- Vercel reads `vercel.json`: build `bun run build`, output `dist`, page rewrites and caching are already set.
- Leave environment variables empty: no variables means demo mode.

3 Press Deploy

In a few minutes you get an address like `rhemlo-app.vercel.app`. Every later push to master redeploys automatically; other branches get their own preview address.

PHASE 0 · CHECK IT

Check the public demo

- Open the address on a phone and a computer. Go through onboarding and one sitting.
- Sign in with a demo account (tiwa@demo.rhemlo.app, password shown on the sign-in sheet).
- Open a lesson in the Library and tap a reference: the KJV text should load.
- You → Downloads: download a quarter, switch the phone to airplane mode, reload. Lessons still open. This only works on the published site, not on your computer's dev server.
- You → Downloads → **Install Rhemlo** adds it to the home screen.

Vercel's free Hobby plan is meant for personal, non-commercial use. A church or ministry service with real users should check the terms and probably use Pro (about $20 per member a month at the time of writing).

Phase 0 done when testers can open the address and the demo works offline.

PHASE 1 · READ FIRST

The live backend is built and tested

Everything the screens need from a server now exists. It was tested locally on PostgreSQL 16 before any real deployment:

Piece	Status
Migrations 0001–0006 (bundle)	Run cleanly on a fresh database; the bundle's 30 behaviour and 63 community checks pass
Migration 0007 (app API)	Sign-up profiles, progress sync, tier leagues, server-marked daily challenge, community, circles, coach mocks, inbox, messages, consent, admin, verification. 72 checks pass
Live adapters in the app	83 calls driven through a PostgREST-like stand-in: every name and parameter matches
Email-link sign-in	The /auth/callback page is in the app
Mail sender	send-mail Edge Function for consent and report emails
Passkeys, push	Demo only; hidden in live mode

Not yet done: running it on a real Supabase project, and a two-phone test on staging. Those are the steps that follow.

PHASE 1 · HOW IT FITS

What runs where

Area	Where it is marked or stored
Practice and mocks	On the device (works offline); synced as one snapshot per learner
Daily challenge	On the server; one attempt; lessons up to this week only; earns verified points
Rooms, friendly matches, Quiz Night	On the server, timed by the server clock; never earn league points
Weekly tier leagues	Verified points only; closed every Monday by a scheduled job
Co-op, circle goals, coach progress	Sittings per day derived from the snapshot (never answers)
Messages	The 0006 safeguards: under-18 rules, guardian consent, blocks, 20 a minute
Reports and admin	Staff roles in platform_staff: super_admin, moderator, support, content_editor

Full detail is in `supabase/README.md` in the repo.

PHASE 1 · STEP BY STEP

Create the Supabase projects

Make **two** projects: **rhemlo-staging** for testing and **rhemlo-prod** for real learners. Do everything on staging first.

1 supabase.com → New project

- Name, a strong database password (store it in a password manager), region near your learners.

2 Note three values

- Project URL: `https://<ref>.supabase.co`
- The **anon public** key: safe in the app.
- The **service_role** key: **secret**. Never put it in the app, in Vercel or in Git.

3 Turn on extensions first

Database → Extensions: enable **pg_cron** and **pg_net** before running the migrations, so the weekly league job and clean-ups schedule themselves.

PHASE 1 · STEP BY STEP

Run the migrations

The seven migrations are already in the repo, in `supabase/migrations`. From the project folder:

```
npx supabase login
npx supabase link --project-ref <staging-ref>
npx supabase db push
```

They run in number order: 0001–0006 from the bundle, then 0007, the app's API.

Want to test on your own computer first? `supabase/tests/run_local.sh` builds a throwaway database, loads both manuals and runs every check.

After it succeeds, open Database → Tables and check that **RLS enabled** shows on every table that holds learner data.

PHASE 1 · STEP BY STEP

Load the manual content

The server needs the question bank so ranked answers are marked on the server. The bundle's own loader does it. Run it from the **bundle** folder with the database connection string (Project Settings → Database → session pooler, port 5432):

```
set DATABASE_URL=postgresql://postgres.<ref>:
  <password>@<pooler-host>:5432/postgres?sslmode=require
set PYTHONPATH=src
python -m rhemlo.cli load --manual manuals/AD2627
python -m rhemlo.cli load-items --manual manuals/AD2627
python -m rhemlo.cli publish --manual manuals/AD2627
(the same three lines for manuals/EN2627)
python -m rhemlo.cli load-bible
```

(Enter DATABASE_URL as one line.) Then add a season for each manual; the SQL is in supabase/README.md.

> The database password in that string is a secret. Type it in your terminal only; never save it in the repo.

PHASE 1 · STEP BY STEP

Staff, coaches and classes

Roles are given in the database by an administrator, never by the app. In the SQL editor, after each person has signed up once:

```
insert into platform_staff (user_id, role)
select id, 'super_admin' from auth.users
where email = 'you@yourchurch.org';
```

- **super_admin**: everything, including the admin console.
- **moderator**: reports and sanctions. **support**: consents and people.
- **content_editor**: the content studio (verification, anomalies, manuals).
- **Coaches**: create a class in `groups` with a join code and add the coach in `group_members` with role coach. Learners join with the code in the app.

Copy-ready SQL for each is in supabase/README.md.

Sign-in settings

Supabase → Authentication:

- **URL configuration**: Site URL = your final address (for example https://rhemlo.app). Add redirect URLs for staging and for Vercel previews, such as `https://*-yourteam.vercel.app/**`.
- **Email provider**: on; require email confirmation.
- **Email templates**: rewrite the confirm and magic-link emails in Rhemlo's voice.
- **Rate limits**: keep the defaults or lower them.
- **Password rules**: at least 8 characters, matching the app.

> Supabase's built-in email is only for testing and is heavily rate-limited. Set up your own sender (Phase 2) before real users arrive.

PHASE 1 · CONNECT THE APP

Switch the app to live

In Vercel → Project → **Settings** → **Environment Variables**, add these three, for the **Preview** environment only at first:

```
VITE_RHEMLO_BACKEND = live
VITE_SUPABASE_URL = https://<staging-ref>.supabase.co
VITE_SUPABASE_ANON_KEY = <staging anon key>
```

- These are baked in when the app is built, so **redeploy** after changing them.
- Push a branch, for example `staging`: Vercel builds a preview on the live staging backend while production stays on the demo.
- The demo banner and the demo accounts disappear automatically in live mode.

> Only the anon key ever goes here. If you ever paste the service_role key into Vercel by mistake, rotate it in Supabase immediately.

PHASE 1 · PROVE IT

Staging test checklist

Use two phones and one computer, with fresh test accounts.

- Sign up, confirm the email, sign out, sign in again.
- Study on phone A; sign in on phone B and see the same progress.
- Go offline mid-sitting, finish, come back online: progress syncs, nothing is lost or doubled.
- Daily challenge: points show pending, then final; refreshing never gives a second attempt.
- League table, private league invite link, friendly match.
- Under-18 account: no messages until guardian consent; adult account: messages work, with the limit of 20 a minute.
- Report a message; act on it as a moderator; the user sees the notice in their inbox.
- Coach sets a mock; a learner submits; the coach sees the score.
- Admin retires a test manual; learners see it as read-only.

Phase 1 done when every line passes on staging, twice, with no console errors.

PATH B · GOOGLE

Host on Firebase Hosting

Use this instead of Vercel if you prefer Google. The backend stays on Supabase.

```
npm install -g firebase-tools
firebase login
firebase init hosting
  public directory: dist
  single-page app: Yes
  GitHub deploys: Yes (optional)
```

Then replace `firebase.json` with:

```
{ "hosting": {
  "public": "dist",
  "ignore": ["firebase.json", "**/.*"],
  "rewrites": [{ "source": "**",
                 "destination": "/index.html" }],
  "headers": [
   { "source": "/sw.js", "headers": [
     { "key": "Cache-Control", "value": "no-cache" }] },
   { "source": "/assets/**", "headers": [
     { "key": "Cache-Control",
       "value": "public, max-age=604800" }] }
  ] } }
```

Real files (assets, data) are served first; every other path falls back to the app. That is the same behaviour as vercel.json.

PATH B · GOOGLE

Build and deploy on Firebase

Firebase only serves files, so the three `VITE_` settings must be present when **you** build. Put them in a `.env.production` file (never commit secrets; the anon key is public but keep the habit):

```
bun install
bun run build
firebase deploy --only hosting
```

- Use a second Firebase site, or `firebase hosting:channel:deploy staging`, for a staging copy.
- Custom domain: Firebase console → Hosting → **Add custom domain**, then add the DNS records it shows. HTTPS is automatic.
- If you enabled GitHub deploys, add the VITE_ values as repository secrets and make the generated workflow run `bun install` and `bun run build`.

> Check Firebase's free (Spark) transfer limit against your traffic. With about 100 MB of artwork and manual data, many first-time visitors can use it up quickly; the Blaze plan is pay-as-you-go.

PATH C · AWS

Host on AWS Amplify

Amplify builds from GitHub and serves through CloudFront. The backend stays on Supabase.

- AWS console → **Amplify** → Create new app → GitHub → pick the repo and branch.
- Add the three VITE_ environment variables in the Amplify app settings.
- Use this build file (add it to the repo as `amplify.yml`):

```
version: 1
frontend:
  phases:
    preBuild:
      commands:
        - npm install -g bun
        - bun install
    build:
      commands:
        - bun run build
  artifacts:
    baseDirectory: dist
    files:
      - '**/*'
  cache:
    paths:
      - node_modules/**/*
```

AWS rewrites and domain

In Amplify → **Rewrites and redirects**, add one rule so app pages load on refresh while real files are served as files:

```
Source:
</^[^.]+$|\.(?!(css|gif|ico|jpg|js|png|txt|svg|
woff|woff2|ttf|map|json|webp|gz|webmanifest)$)
([^.]+$)/>
Target: /index.html
Type: 200 (Rewrite)
```

(Enter the source as one line with no breaks.)

- Custom domain: Amplify → **Domain management**. If your domain is in Route 53, it is configured for you; otherwise add the records it shows.
- For a staging copy, connect a second branch (for example staging) with staging values for the VITE_ settings.
- Advanced alternative: S3 bucket + CloudFront distribution with a 403/404 → /index.html error rule. More control, more setup.

PHASE 2 · POLISH

Your domain and HTTPS

1 Buy the name

For example rhemlo.app or rhemlo.org. Turn on auto-renew and registrar lock.

2 Point it at your host

- Vercel: Project → Settings → Domains → add it, then copy the records shown (usually an A record for the bare domain and a CNAME for www).
- Firebase or Amplify: the same idea in their consoles.

3 Wait for HTTPS

Certificates are issued automatically, often within an hour. Redirect www to the bare domain (or the reverse) so there is one address.

4 Update Supabase

Set the Site URL and redirect URLs to the final domain, or email links will point to the old address.

PHASE 2 · POLISH

Email that arrives

Sign-in links, guardian consent requests and weekly reports must reach inboxes, not spam.

- Create a Resend account and add your domain. Add the **SPF**, **DKIM** and **DMARC** records it gives you.
- Supabase → Auth → **SMTP**: enter Resend's SMTP details. This sends sign-in and confirmation emails.
- Deploy the mail sender for everything else:

```
npx supabase functions deploy send-mail
  --no-verify-jwt
npx supabase secrets set RESEND_API_KEY=...
  MAIL_FROM="Rhemlo <no-reply@mail.rhemlo.app>"
  CRON_SECRET=<random>
```

Then schedule it every five minutes with the SQL in supabase/README.md. Send yourself a sign-in link and a consent request to check both arrive.

PHASE 2 · MUST DO

Security and privacy checklist

- Every table with learner data has RLS on, and each rule is tested with two different accounts.
- Only the anon key is in the app; the service_role key lives only on your computer or a server.
- Two-factor sign-in on GitHub, Vercel/Firebase/AWS, Supabase, the registrar and the email provider.
- Supabase backups: on the paid plan, turn on daily backups (point-in-time recovery if budget allows) and practise one restore on staging.
- Children’s data: birth year only, guardian consent recorded, under-18 messaging rules on (already built).
- Publish a privacy policy and terms. For Nigerian users, follow the Nigeria Data Protection Act 2023; take local advice.
- Name a person who handles safety reports and data requests, with a response time.

> Do not invite real learners until every line here is ticked.

PHASE 2 · POLISH

Monitoring

What	Tool	Why
Front-end errors	Sentry (free tier)	See crashes on learners' phones, with the screen they were on
Uptime	Better Stack, UptimeRobot or similar	Get a message if the site or Supabase is down
Database health	Supabase reports and logs	Slow queries, connection count, storage
Traffic	Vercel Analytics, Firebase or Amplify metrics	Visitors and Web Vitals, privacy-friendly

Watch the first two weeks closely, especially Sunday evenings after class, when everyone studies at once.

BUDGET

Rough monthly costs

Indicative only, at the time of writing. Check each provider's pricing page.

Item	Start	Growing
Web host (Vercel Pro / Firebase Blaze / Amplify)	About $0–20	About $20–60, mostly bandwidth
Supabase	Free for staging; Pro about $25 for production	Higher compute add-ons as users grow
Domain	About $10–20 a year	Same
Email (Resend / Postmark)	Free tier to about $15	By volume
Sentry, uptime	Free tiers	About $0–30

Supabase's free projects pause after about a week without activity. Fine for staging, never for production.

The biggest bandwidth cost is first-time visitors downloading artwork and manual data. Compressed images and long cache times (already set) keep it down.

LAUNCH DAY

Go-live checklist

- Production Supabase project created, migrations run, content loaded, cron jobs scheduled.
- Production VITE_ settings added for the **Production** environment; redeployed.
- Your own account signs up, studies, syncs to a second device and appears in a league.
- Admin, coach and steward roles granted to the right people in the database.
- Email arrives; sign-in links open the right domain.
- Error and uptime alerts reach your phone.
- A short announcement to testers, then one parish or class, then everyone.

> Launch in waves: one class for a week, then a province, then everyone. Each wave shows problems while they are still small.

IF SOMETHING GOES WRONG

Rolling back

- **Bad release**: Vercel → Deployments → choose the last good one → **Promote to Production**. It takes seconds. Firebase and Amplify have the same “roll back to previous” buttons.
- **Backend trouble**: put the site in maintenance with a banner, or temporarily set `VITE_RHEMLO_BACKEND` back to demo and redeploy (learners then study on their own device only).
- **Data mistake**: restore the Supabase backup into staging first, check it, then restore production.
- **Leaked key**: rotate it in Supabase at once, update Vercel, redeploy.

> Write the names of who can do each of these, and keep this page where they can find it.

YOUR NEXT STEP

The journey from here

#	Step	Rough time
1	Phase 0: put the demo on Vercel for testers	An afternoon
2	Create Supabase staging; enable pg_cron and pg_net; db push	1 hour
3	Load both manuals and the KJV with the bundle loader; add a season	1 hour
4	Staff roles, a test class, sign-in and SMTP settings, mail sender	Half a day
5	Preview deploy on live staging; staging test checklist, twice	3–5 days
6	Domain, security checklist, monitoring	2–3 days
7	Production project (repeat 2–4) and launch in waves	1–2 weeks

The server code is written and tested locally. What remains is setting it up on real accounts and testing it with real phones.

WORDS USED

Glossary

Term	Meaning
Static app	An app made only of files the browser downloads; no server code of its own
CDN	A worldwide network of servers that delivers files from close to each visitor
Supabase	A hosted Postgres database with sign-in and an automatic API
Migration	A versioned SQL file that creates or changes database tables
RLS	Row-level security: rules saying who may read or change each row
RPC	A database function the app calls by name
anon key	Public key for the app; safe only because of RLS
service_role key	Master key that ignores RLS; secret
Environment variable	A setting given to the build, such as VITE_SUPABASE_URL
Staging	A private copy for testing before production
pg_cron	Postgres add-on that runs jobs on a schedule
SPF / DKIM / DMARC	DNS records proving your emails are really from you